

JWT Security and Secrets Management Guide

This document provides best practices for securely managing JWT signing keys and secrets in .NET 8 applications. It includes steps for configuring JWT authentication, generating secure keys, and protecting them using User Secrets, Environment Variables, or Azure Key Vault.

1. What Are Signing Credentials and Security Keys?

Signing credentials tell the API what algorithm and key to use when creating or validating JWT tokens. The key (SecurityKey) is the actual secret used to sign tokens. Always use strong keys (at least 256 bits for HS256).

2. Generating a Secure Key (PowerShell)

Run the following PowerShell command to generate a strong 256-bit key:

```
# Generate a 32-byte random key (Base64 encoded)
$bytes = New-Object Byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
$key = [Convert]::ToBase64String($bytes)
Write-Output "Generated JWT Key:"
Write-Output $key
```

3. appsettings.json Example

```
"Jwt": {
  "Issuer": "StudentEnrollmentApi",
  "Audience": "StudentEnrollmentApiUsers",
  "Key": "THIS_IS_A_SUPER_SECRET_KEY_CHANGE_IT_1234567890!",
  "ExpiresInMinutes": 120
}
```

4. Using .NET User Secrets (Local Development)

User Secrets allow you to store secrets outside the project folder. This prevents sensitive data from being committed to GitHub. Use this for local development.

```
# Step 1: Initialize secrets for your project
dotnet user-secrets init

# Step 2: Add JWT secrets
dotnet user-secrets set "Jwt:Key" "YOUR_GENERATED_KEY"
dotnet user-secrets set "Jwt:Issuer" "StudentEnrollmentApi"
dotnet user-secrets set "Jwt:Audience" "StudentEnrollmentApiUsers"
```

5. Using Environment Variables (Production)

```
# PowerShell (Windows)
$env:Jwt__Key="YOUR_PRODUCTION_SECRET_KEY"
$env:Jwt__Issuer="StudentEnrollmentApi"
$env:Jwt__Audience="StudentEnrollmentApiUsers"

# Bash (Linux/macOS)
export Jwt__Key="YOUR_PRODUCTION_SECRET_KEY"
export Jwt__Issuer="StudentEnrollmentApi"
```

```
export Jwt__Audience="StudentEnrollmentApiUsers"
```

6. Security Best Practices

1. Never store secrets in appsettings.json for production. 2. Use User Secrets for local development. 3. Use environment variables or Azure Key Vault for production. 4. Rotate keys periodically. 5. Do not commit sensitive files to GitHub. 6. Ensure .gitignore excludes secrets.json and private keys.

7. Summary

- Signing credentials define how JWTs are signed and validated.
- Security keys are the secret material that ensures tokens can't be forged.
- Manage secrets securely using User Secrets, Environment Variables, or Key Vault.